# Threads: sincronización y comunicación

Una analogía con dos personas, una habitación y una sola llave

**synchronized · wait() · notify() · notifyAll()**

# La situación: productor y consumidor




A coloca productos. B los retira. La caja admite un solo producto.

# ¿Qué representa cada elemento?

## Personas

Dos Threads que ejecutan tareas independientes.

## Habitación y caja

Objeto compartido y su estado.

## Llave

Monitor o bloqueo del objeto.

## Reglas de acceso

Sólo una persona posee la llave a la vez.

# Las cuatro instrucciones

## synchronized

Adquiere la llave para entrar. Si otro la tiene, espera.

## wait()

Se suspende y deja la llave disponible.

## notify()

Avisa a un Thread que está esperando en wait().

## notifyAll()

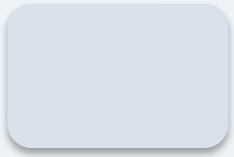
Avisa a todos los Threads que esperan en ese objeto.

# 1 · A entra primero

A

Dentro

OBJETO COMPARTIDO



**Caja vacía**

**Monitor: A**

B

Fuera

A adquiere el monitor mediante synchronized. Como la caja está vacía, puede producir.

# 2 · A coloca un producto

A

Dentro

OBJETO COMPARTIDO



**Caja llena**

**Monitor: A**

B

Bloqueado

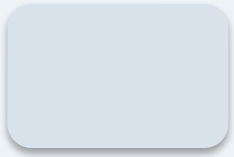
A coloca el producto, notifica y, al salir del método, libera el monitor.

# 3 · B entra y consume

A

Fuera

OBJETO COMPARTIDO



**Caja vacía**

**Monitor: B**

B

Dentro

B adquiere el monitor y retira el producto. La caja vuelve a quedar vacía.

# 4 · B intenta consumir de nuevo




B encuentra la caja vacía: ejecuta wait(), se suspende y libera el monitor.

# 5 · A vuelve a producir

A

Dentro

OBJETO COMPARTIDO



**Caja llena**

**Monitor: A**

B

En wait()

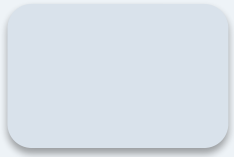
A coloca otro producto y ejecuta notifyAll(). B es avisado, pero A todavía tiene la llave.

# 6 · B retoma su trabajo

A

Fuera

OBJETO COMPARTIDO



**Caja vacía**

**Monitor: B**

B

Dentro

A libera el monitor. B lo recupera, reevalúa el while y puede consumir.

# Dos esperas diferentes

## Bloqueado por synchronized

Quiere entrar, pero otro Thread posee el monitor. No ejecutó wait().

## Suspendido por wait()

Ya estaba dentro; se suspende voluntariamente y libera el monitor.

**No son equivalentes: esperar por la llave ≠ dejar la llave y esperar una condición.**

# ¿Qué ocurre al ejecutar notifyAll()?

1. Despierta a los Threads que hicieron wait() sobre ese objeto.
2. No entrega inmediatamente la llave.
3. El emisor conserva el monitor hasta liberarlo.
4. Los despertados compiten por recuperarlo.
5. Quien lo consigue retoma después de wait() y reevalúa el while.

# Código Java: recurso compartido

```
class Caja
{
    private boolean disponible = false;
    public synchronized void producir() throws InterruptedException
    {
        while (disponible)
           wait();
        disponible = true;
        notifyAll();
    }
    public synchronized void consumir() throws InterruptedException
    {
        while (!disponible)
           wait();
        disponible = false;
        notifyAll();
    }
}
```

# ¿Por qué while y no if?

**Despertar no garantiza que la condición ya sea verdadera.**

Entre la notificación y la recuperación del monitor, otro Thread puede cambiar el estado de la caja.

El while vuelve a comprobar la condición y, si corresponde, ejecuta wait() nuevamente.

# Para recordar

### synchronized

Protege el acceso al objeto compartido.

### wait()

Suspende al Thread y libera el monitor.

### notifyAll()

Avisa a los que esperan; no les entrega el monitor.

**Idea central: exclusión mutua + coordinación por condiciones.**

# Preguntas para discutir en clase

¿Puede B ejecutar consumir() mientras A está dentro de producir()?

¿Puede A ingresar cuando B ejecutó wait()?

¿B continúa inmediatamente al recibir notifyAll()?

¿Por qué conviene usar while en lugar de if?

# Referencia de la cátedra

Material de base: “Clase 10 – Concurrencia” (2026).

Especialmente: métodos synchronized, comunicación entre Threads y ejemplo Productor–Consumidor (páginas 31–58).

La simulación presenta un orden posible, no un orden garantizado por la JVM.